

How to Choose a Replication Strategy



BYTEBYTEGO

SEP 7, 2023 • PAID



100



4

Share



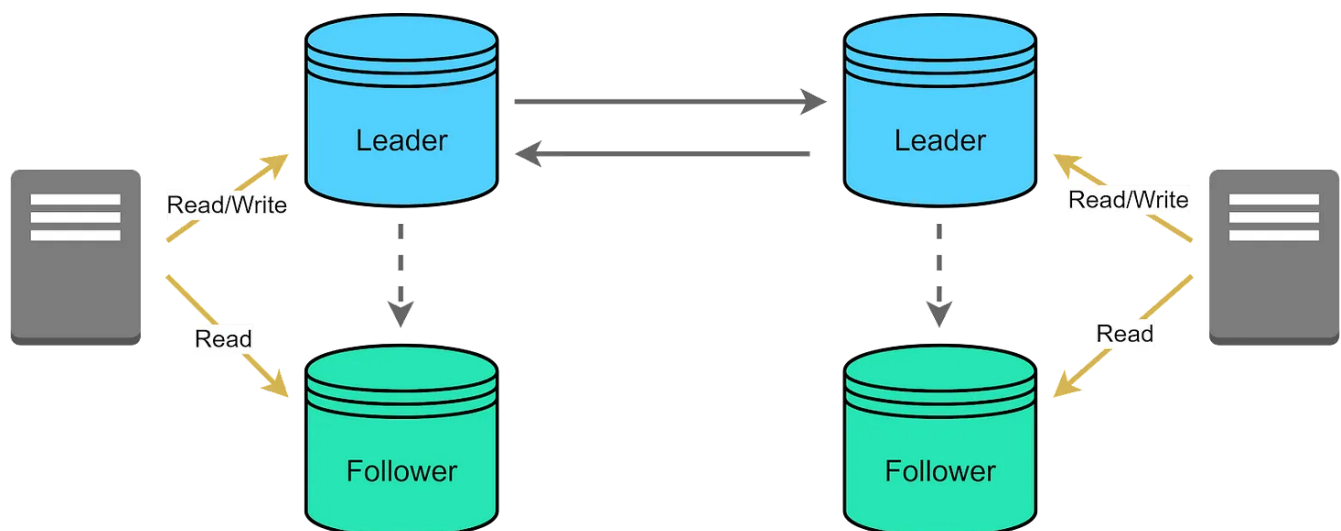
In the last issue, we kicked off a 2-part series exploring common data replication strategies. We learned about the leader-follower model - its synchronous and asynchronous variations, consistency considerations, failure handling, and more.

In this issue, we'll examine two alternative approaches - multi-leader and leaderless replication. We'll contrast their designs, dive into how they work, and see the types of use cases where they excel.

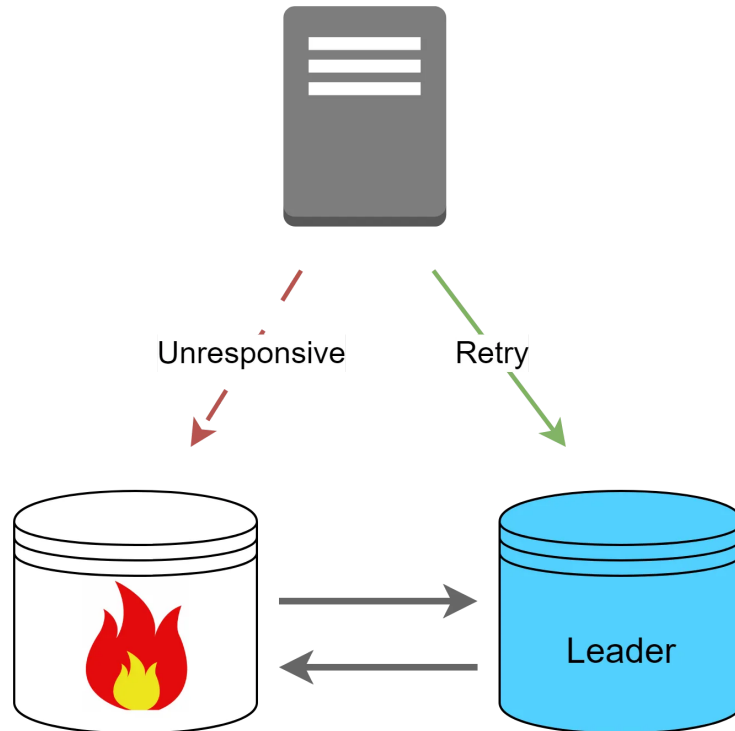
By the end, you'll understand the core replication models and how to select the right strategy based on your system needs and constraints. Let's jump back in where we left off last week.

Multi-Leader Replication

Multi-leader replication, sometimes called leader-leader replication, involves the use of multiple primary nodes, also known as leaders, each capable of receiving and processing write requests. These leaders replicate data between each other to stay up to date. Each leader may also have follower replicas for read scaling.



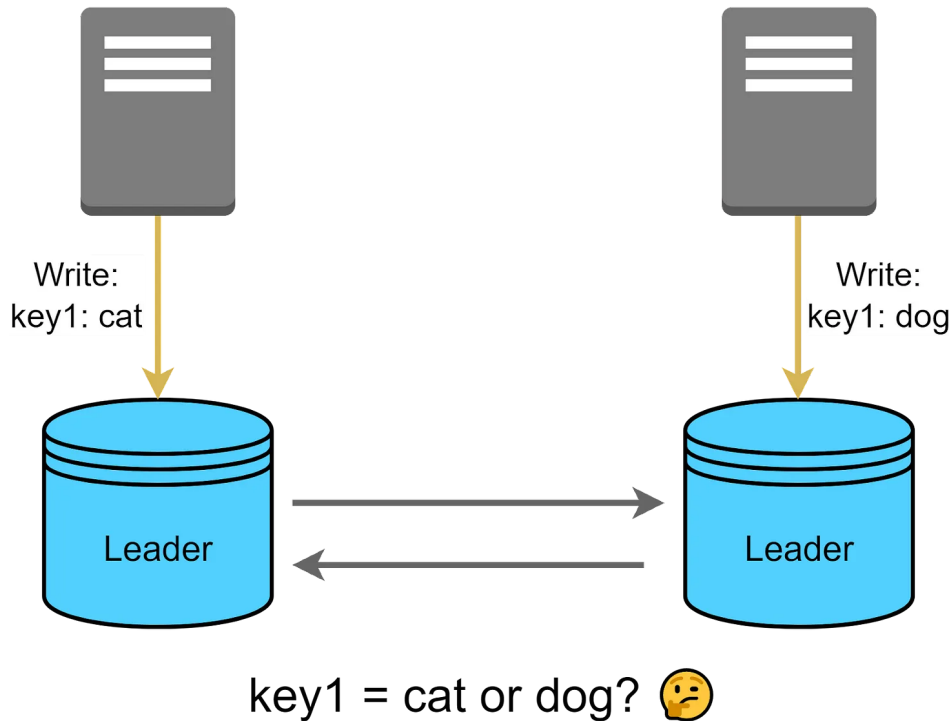
The primary advantage of this model is increased write availability. With multiple active leaders, failure of one node doesn't disrupt writes - other leaders continue handling write requests. This improves upon leader-follower designs where a failed leader halts writes until a new leader is available.



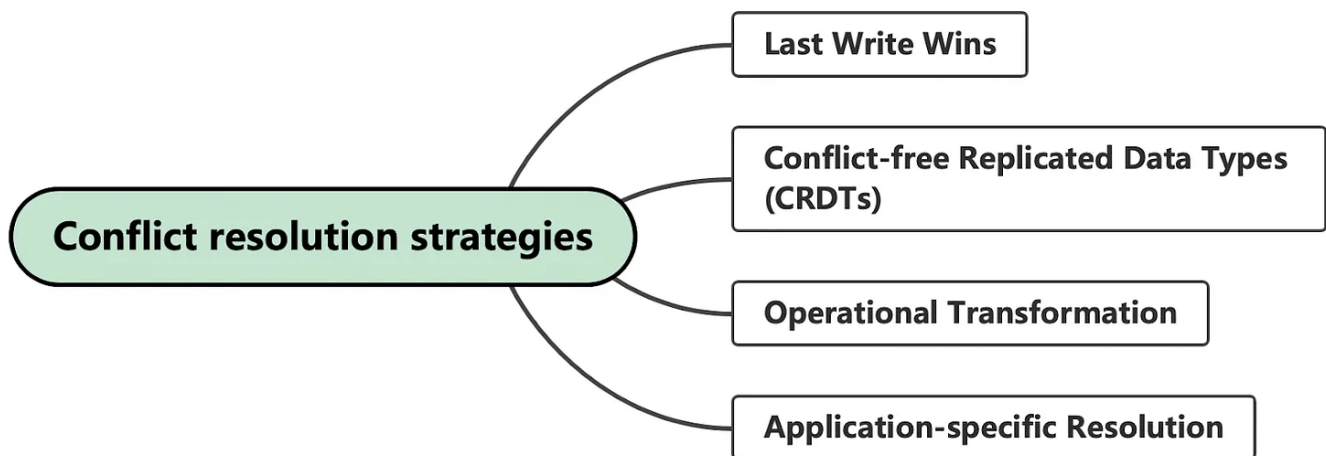
However, multi-leader replication comes with its own set of challenges. For instance, with multiple leaders handling write requests, conflicting changes may occur when leaders modify the same data concurrently.

Managing Conflict

Conflicts are a natural outcome in multi-leader replication models given that multiple leaders can perform write operations simultaneously. Effective conflict management is a complex task, but it is crucial for ensuring data consistency and integrity.



Here are some commonly employed conflict resolution strategies.



Last Write Wins

This is a straightforward method where the most recent change takes precedence. While easy to implement, it risks discarding important updates.

Conflict-free Replicated Data Types (CRDTs)

CRDTs allow for seamless reconciliation of conflicting changes by merging them. CRDTs come in various types for different kinds of data like counters, sets, and lists, and automatically resolve conflicts without requiring a separate conflict resolution process.

Operational Transformation

Operational transformation is often used in real-time collaborative applications. It takes the operation itself into account, not just the state of the data. This method is complex to implement but offers fine-grained control.

Application-specific Resolution

In some cases, conflict resolution logic can be pushed to the application level. The application can employ domain-specific rules or even involve human intervention for resolving conflicts.

Data Partitioning

Another alternative is to partition data across multiple leaders to minimize conflicting changes. However, implementing cross-partition transactions requires careful coordination, and potential hot spots on busy data partitions need to be managed effectively. It's worth noting that this strategy can reduce the overall write throughput across the cluster.

Replication Lag and Inconsistent Reads

As with leader-follower replication, multi-leader systems are susceptible to replication lag and inconsistent reads. They cause temporary inconsistencies between leaders until updates fully propagate. Applications must be designed with this in mind.

Use Cases

What are some use cases for multi-leader replication? For applications that have users globally, multi-leader replication can reduce the latency for end-users by allowing them to interact with a nearby leader node.

Systems that cannot afford downtime, such as financial transaction platforms, can benefit from having multiple leaders. Even if one goes down, operations continue.

For applications with heavy write loads, distributing the write operations across multiple leaders can prevent any single node from becoming a bottleneck.

Tradeoffs and Challenges

In essence, multi-leader replication is particularly useful for applications that prioritize high write availability, fault tolerance, and globally-distributed data accessibility. Many modern

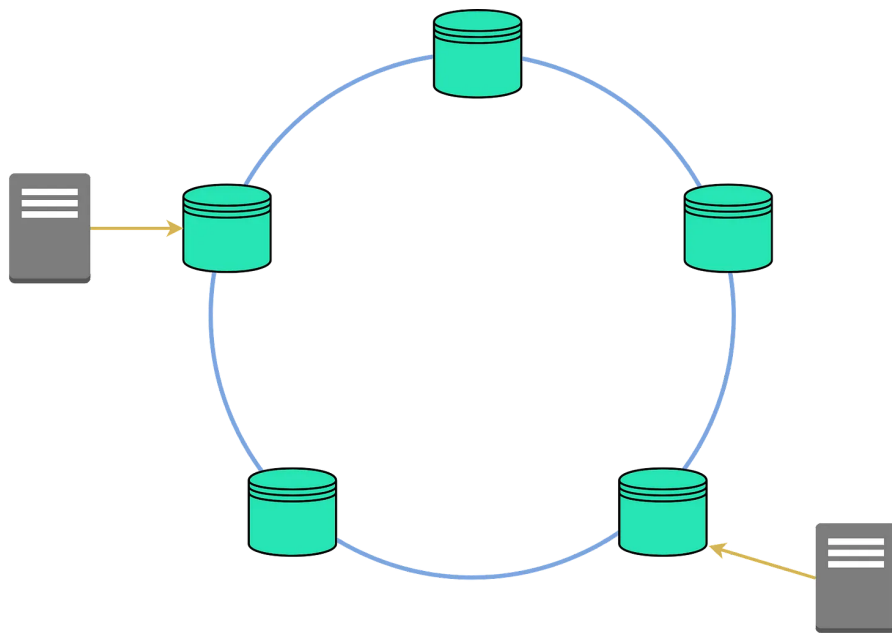
databases can leverage this replication strategy, either natively, or with an extension, with varying degrees of success.

Multi-leader replication provides high availability but requires careful design around consensus, conflict detection, and resolution mechanisms. When implemented well, it can be a powerful approach for maximizing write throughput and availability.

In the next section, we'll explore the leaderless replication model which takes a different approach.

Leaderless Replication

Leaderless replication takes a quorum-based approach. This concept may sound a bit strange, especially when we've just spent some time discussing models that operate under a clear hierarchy. In a leaderless system, any node in the network has the authority to accept write operations. The absence of a single leader fundamentally changes the dynamics of our system.



Quorum Writes and Reads

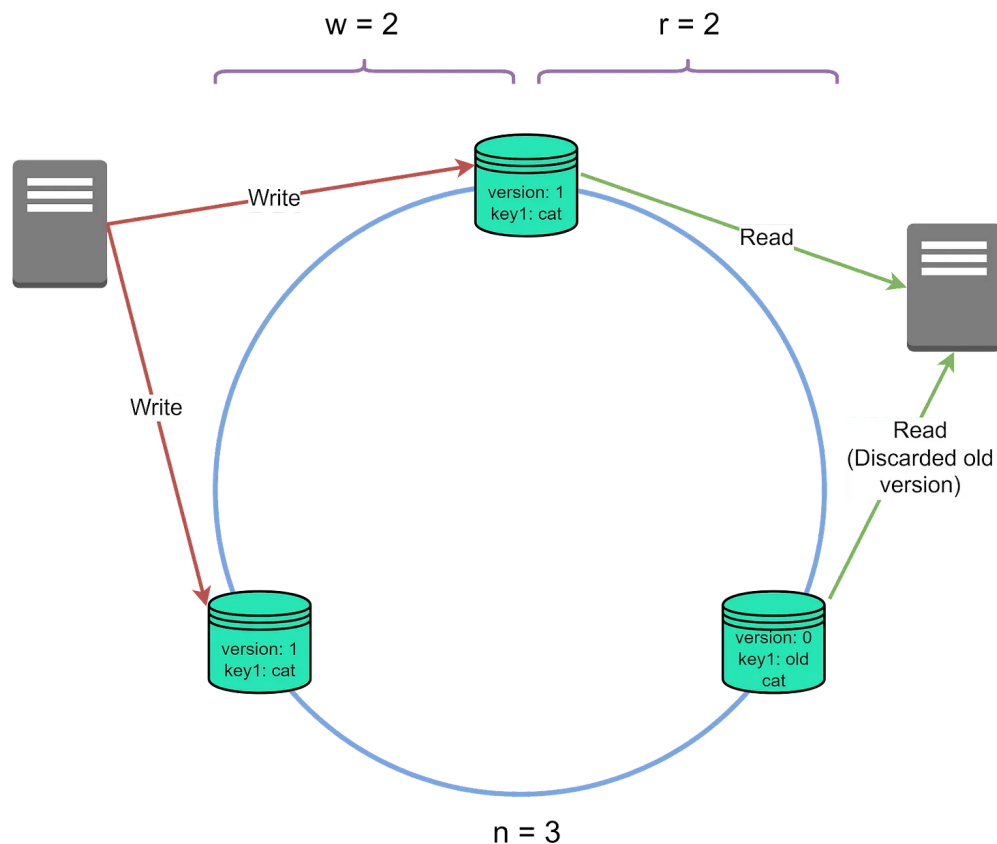
Now, let's start with a key concept that underpins leaderless replication: 'quorum writes and reads'. In a system without a leader, we don't rely on any single node to validate a read or write operation. Instead, we aim for consensus among a certain number of nodes. This number is called the 'quorum'. Using a quorum approach balances high availability with data accuracy, since we no longer require full consensus across all nodes.

In this system, we use three important values.

- 'n' is the total number of nodes in our system.
- 'W', the write quorum, is the minimum number of nodes that need to agree for a write to be considered successful.
- 'r', the read quorum, is the minimum number of nodes that need to agree for a read to be valid.

For strong consistency, a general guideline is to have $w + r > n$. It ensures that any read overlaps with any write and returns the most recent value.

For example, imagine a system with 3 nodes ($n=3$). If we configure w to 2, that means we need two out of three nodes to acknowledge a write request before it is deemed successful. If one of the nodes went down, the write operations could still continue. This idea works similarly for reads. If r is set to 2, the read operation would query 2 nodes and return the most recent data between the two.



The closer $w + r$ is to n , the greater the chances of reading up-to-date data. In the extreme case, if $r = n$, we are guaranteed to read the up-to-date data, though it might take a while for all the nodes to agree on the correct value. In the other extreme, if $w = n$, we will always read the latest value, no matter what r is. And of course, a high value of w slows down write operations, as we have to wait for the data to be acknowledged by all nodes.

Benefits and Drawbacks

Leaderless replication offers several advantages, primarily higher availability. With no single leader acting as a potential point of failure, the system can withstand individual node failures without significant impact. It also sidesteps the need for intricate leader election procedures, which can be complex to implement and maintain.

But like all systems, leaderless replication isn't perfect. It presents us with a significant challenge: conflict resolution. In the absence of a single source of truth, we could end up with different nodes holding different versions of the same data. This discrepancy raises a question: which version should we keep?

Conflict Resolution Strategies

Thankfully, we have a few strategies to handle these conflicts. One approach is to use timestamps or version numbers. In this method, the data with the most recent timestamp or the highest version number is considered the truth. Another way is to use application-specific logic. This strategy involves setting rules within our application that decide which version is correct when a conflict arises. For example, a rule could be to keep the version with the higher numerical value for a particular field.

To see leaderless replication in action, we can look at systems like Apache Cassandra. Cassandra uses a quorum-based approach, where both reads and writes require a quorum of acknowledgments across nodes. Writes are sent to all replica nodes, while reads only query enough nodes to meet the read quorum. Timestamps are used for conflict resolution. This design allows Cassandra to provide high availability without the need for a leader. It allows tunable consistency levels, from eventual to strong, based on the quorum settings. Although generally leaderless, Cassandra uses a leader-based approach for specific features like lightweight transactions.

In the next section, we're going to take all of this knowledge and apply it. We'll discuss how to choose the right replication strategy based on our specific needs and constraints.

Choosing a Replication Strategy

Here is a summary comparison of the key replication strategies:

Strategy	Consistency	Write Availability	Complexity
Leader-follower	Strong	Low (depends on leader)	Low
Multi-leader	Weak	High	High
Leaderless	Weak	High	Moderate

Several key factors should be evaluated when selecting a replication strategy. The best approach depends on the specific system architecture and application requirements.

System Size and Complexity

The size and complexity of the system are important factors. Large and complex systems make replication more difficult. Synchronous replication can slow down response times when copying large amounts of data across nodes. Smaller, simpler systems may not face these issues. Complex data structures also pose challenges for conflict resolution. Some replication strategies handle complexity better than others. Additionally, more complex systems may require more intricate monitoring and conflict-resolution mechanisms.

Consistency Needs

How much consistency does the application require? Applications demanding strong consistency may need synchronous replication to ensure replicas are synchronized. However, this impacts performance and availability. Applications tolerating some inconsistency can use asynchronous replication for better response times and availability despite replica lag.

Geographic Distribution

Consider geographic distribution and its impact on network latency. Systems spanning multiple regions struggle with synchronous replication's latency demands. Leaderless and multi-leader replication better tolerate network delays across global deployments. They maximize availability despite dispersed infrastructure. Furthermore, be aware that data

replication across international boundaries may be subject to different data protection regulations.

Read/Write Workloads

Workload nature influences replication choice. Read-heavy systems suit leader-follower replication. Read requests can be distributed across follower replicas to balance load and improve response times. Write-heavy systems are better served by multi-leader or leaderless replication where writes are parallelized.

Replication Factor

The replication factor balances cost and durability. Higher factors increase resilience through more copies but also raise storage and management overhead. A factor of 3 is typical for good availability without excessive overhead, although the optimal factor can vary based on specific system requirements..

There are no perfect choices, only better trade-offs for each unique system architecture and application. How do these factors come into play for real-world systems?

Replication Strategy Examples

Here are some example scenarios demonstrating how these factors can influence replication strategy selection in real-world systems:

Online Retail Application

- Large product catalog makes database large and complex
- Requires strong consistency for order processing
- Deployed across multiple geographic regions
- Mostly read operations for product browsing

Recommended strategy: Leader-follower replication to handle large data, ensure strong consistency, and distribute reads across regions.

Gaming Application

- Player data is relatively simple
- Highly sensitive to latency for good gameplay

- Read and write heavy for game state updates
- Operates within a region for low latency

Recommended strategy: Multi-leader replication for fast reads/writes in a single region

Ride Sharing Application

- Medium complexity data
- Requires eventual consistency for high availability
- Write-heavy workloads for trip updates
- Cost sensitive infrastructure

Recommended strategy: Leaderless replication for efficient writes. Conflict resolution provides eventual consistency at low infrastructure cost.

Evaluating system characteristics against application requirements leads to replication strategies optimized for performance, scalability and reliability. There are no perfect choices, only better trade-offs for each unique architecture and application. Keep in mind that changing business needs or scaling might require a re-evaluation of your chosen replication strategy.

Conclusion

Replication is a critical technique for building robust and highly available distributed systems. In this issue, we explored popular replication strategies and their trade-offs.

Leader-follower replication offers strong consistency but represents a single point of failure. Multi-leader improves write availability but introduces complexity. Leaderless prioritizes availability by removing leaders entirely.

There is no universally ideal replication strategy. The best approach depends on factors like consistency needs, infrastructure, costs, and workload patterns. Effective system designers understand these trade-offs and choose strategies aligned with application requirements and business needs.

This article provided a foundation for making informed decisions about replication. We covered key concepts and real-world examples applicable to distributed systems. However, there is always more complexity to uncover around these strategies. The landscape is

dynamic, and new methods and technologies continue to evolve. Use this guide as a starting point and stay informed of new developments in this area to design robust, highly available distributed systems.



100 Likes · 4 Restacks

Comments



Write a comment...

© 2023 ByteByteGo · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great writing